

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт

Про виконання лабораторної роботи №2
«Реалізація алгоритмів в функціональному стилі»

Виконала:

Студентка групи ФЕП-11

Матла Діана Ярославівна

Перевірив:

Асистент Левуш Павло Назарович

Львів 2026

Мета: Реалізувати альтернативний метод перевірки приналежності точки трикутника за допомогою методу векторного добутку.

Хід роботи

Файли та готовий код було відправлено на github за посиланням:

<https://github.com/Diana-Matla/OOP/tree/main/lab%202>

Було створено програму, яка перевіряє приналежність точок трикутника за допомогою методу векторного добутку та методу площ.

```
points.h > ...
1  #ifndef POINTS_H
2  #define POINTS_H
3
4  struct Point {
5      double x, y;
6  };
7
8  struct Triangle {
9      Point A, B, C;
10
11      double triangleArea() const;
12      void PositionHeron(Point P) const ;
13      void Position(Point P) const;
14  };
15
16  double distance(const Point &A, const Point &B);
17  double determinant(Point A, Point B, Point P);
18  bool onSegment(Point A, Point B, Point P);
19  int sign(double n);
20  void inputPoint(Point &p);
21  int result();
22
23 #endif
```

points.h

```

main.cpp > main()
1  #include "points.cpp"
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      result();
7      return 0;
8  }

```

Main.cpp

```

points.cpp > triangleArea() const
1  #include "points.h"
2  #include <cmath>
3  #include <iostream>
4  #include <algorithm>
5  using namespace std;
6
7  //Відстань між точками
8  double distance(const Point &A, const Point &B) {
9      return sqrt(pow(B.x - A.x, 2) + pow(B.y - A.y, 2));
10 }
11
12
13 //Площа трикутника
14 double Triangle::triangleArea() const {
15     double a = distance(A, B);
16     double b = distance(B, C);
17     double c = distance(C, A);
18
19     double p = (a + b + c) / 2.0;
20     return sqrt(p * (p - a) * (p - b) * (p - c));
21 }
22
23
24 //Позиція точки відносно трикутника(за Героном)
25 void Triangle::PositionHeron(Point P) const{
26     double S = triangleArea();
27
28     Triangle T1 = {P, A, B};
29     Triangle T2 = {P, B, C};
30     Triangle T3 = {P, C, A};
31
32     double S1 = T1.triangleArea();
33     double S2 = T2.triangleArea();
34     double S3 = T3.triangleArea();
35 }

```

```

        if (fabs(S - (S1 + S2 + S3)) < 1e-9) {
            if (S1 < 1e-9 || S2 < 1e-9 || S3 < 1e-9)
                cout << "on the edge (Heron method)" << endl;
            else
                cout << "inside the triangle (Heron method)" << endl;
        } else {
            cout << "outside the triangle (Heron method)" << endl;
        }
    }

    double determinant(Point A, Point B, Point P) {
        return (B.x - A.x)*(P.y - A.y) - (B.y - A.y)*(P.x - A.x);
    }

    bool onSegment(Point A, Point B, Point P) {
        return (P.x >= min(A.x, B.x) && P.x <= max(A.x, B.x) &&
            P.y >= min(A.y, B.y) && P.y <= max(A.y, B.y));
    }

    int sign(double n) {
        if (fabs(n) < 1e-9) return 0;
        return (n > 0) ? 1 : -1;
    }
}

```

```

int sign(double n) {
    if (fabs(n) < 1e-9) return 0;
    return (n > 0) ? 1 : -1;
}

void Triangle::Position(Point P) const{
    int sign1 = sign(determinant(A, B, P));
    int sign2 = sign(determinant(B, C, P));
    int sign3 = sign(determinant(C, A, P));

    if (((sign1>0) && (sign2>0) && (sign3>0)) || ((sign1<0) && (sign2<0) && (sign3<0))) cout<<"inside the triangle(Vector)"<<endl;

    else if ((sign1 == 0 && onSegment(A, B, P)) ||
        (sign2 == 0 && onSegment(B, C, P)) ||
        (sign3 == 0 && onSegment(C, A, P))) {
        cout << "on the edge(Vector)" << endl;
    }
    else cout<<"outside the triangle(Vector)"<<endl;
}

```

```

void inputPoint(Point &p) {
    while (true) {
        cin >> p.x >> p.y;

        if (!cin.fail()) break;

        cout << "Invalid input. Enter two numbers: ";
        cin.clear();
        cin.ignore(1000, '\n');
    }
}

```

```

int result(){
    int amount=0;
    Triangle triangle;
    Point points[100];

    cout << "Enter coordinates A (x1, y1): ";
    inputPoint(triangle.A);

    cout << "Enter coordinates B (x1, y1): ";
    inputPoint(triangle.B);

    cout << "Enter coordinates C (x1, y1): ";
    inputPoint(triangle.C);
}

```

```

if (triangle.triangleArea() < 1e-9) {
    cout<<"Triangle is degenerated"<<endl;
}
else {
    cout<<"Triangle is not degenerated"<<endl;
}

cout<<"Enter amount of points you want to add"<<endl;
cin>>amount;

if(amount>100){
    cout<<"Too much points"<<endl;
    return 1;
}

for (int i=0; i<amount; i++){
    cout<<"Enter coordinates for "<<i+1<<" point (x,y):"<<endl;
    inputPoint(points[i]);
}

for (int i = 0; i<amount; i++){
    cout<<"The Point "<< i+1 <<" is "<<endl;
    triangle.Position(points[i]);
    triangle.PositionHeron(points[i]);

    cout << endl;
}
return 0;
}

```

Висновок: У ході виконання лабораторної роботи було реалізовано альтернативний підхід до визначення приналежності точки трикутнику за допомогою методу векторного добутку, що дозволяє ефективно враховувати взаємне розташування точки відносно сторін фігури. Додатково було впроваджено перевірку виродженості трикутника через обчислення його площі, що забезпечує коректність роботи алгоритму навіть у граничних випадках. Програма була вдосконалена для обробки довільної кількості точок, що підвищує

її універсальність та практичну цінність. Також реалізовано введення координат трикутника та точок користувачем, що робить програму більш гнучкою у використанні. Окрему увагу приділено обробці ситуацій, коли точка лежить на межі трикутника, що забезпечує повноту та точність отриманих результатів.